T11b Yarn Spinner (optional)

CRE132 worksheet

What you'll make

This sample is **optional**. It takes T11's village further with **Yarn Spinner**, a free tool for writing game dialogue. T11's Talkable was a list of lines. Real conversations need **choices**, need to **remember** what happened, and need to **do** things in the game.

The blacksmith introduces himself the first time and says "Back again?" after that. He offers you the gold key for 3 coins, and you choose what to say. With too few coins he tells you how many you have; collect 3 from around the square, buy the key, and the coins leave your inventory as the key arrives. The villager's advice changes once you've bought it, and the key opens the pen. The dialogue is written in `.yarn` files that read like a play script, so a writer can work on it without touching C#.



Figure 1: The finished scene: T11's village with coins on the ground, a pen with gems behind a locked door, and the blacksmith offering a choice.

Before you start

1. Unzip `CRE132_T11b_YarnSpinner_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_T11b_YarnSpinner_Starter` folder. Open it with Unity **6000.3.11f1**. The first time it opens, Unity downloads the Yarn Spinner package from GitHub, so you need to be online.
2. Open the scene `Assets/_game/Scenes/TopDown/T11b_YarnSpinner`.
3. Press **Play**. You can walk (WASD or arrows), swing the sword (J) and pick up coins (T09's inventory, top right). The **Locked Door** to the pen stays shut without the gold key (T10). Every **E** prompt is showing, and nobody talks yet.

Yarn Spinner is already installed (it's listed in `Packages/manifest.json`). To add it to your own project: **Window > Package Manager**, **+**, **Install package from git URL**, and paste `https://github.com/YarnSpinnerTool/YarnSpinner-Unity.git#v3.2.8`.

Two Yarn things are already in the project:

- **`Assets/_game/Dialogue/T11b/Village.yarnproject`**, a **Yarn Project**. It collects every `.yarn` file in its folder and turns them into dialogue the game can run.
- **Dialogue System** in the Hierarchy, Yarn's own prefab. It has a **Dialogue Runner** (plays the dialogue; its **Yarn Project** is set to **Village**), a **Line Presenter** (the box with the name and the typing text), an **Options Presenter** (the choice buttons) and a **Line Advancer** (set to **Input Actions** with our **Interact** action, so E moves the lines on as in T11).

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

Making a Yarn script: right-click in `Assets/_game/Dialogue/T11b/` and choose **Create > Yarn Spinner > Yarn Script**, type the name and press Enter. Double-click it to open it in your code editor and replace everything. The free **Yarn Spinner** extension for VS Code colours and checks `.yarn` files. The finished `Sign.yarn`, `Blacksmith.yarn` and `Villager.yarn` are in the same folder of `CRE132_T11b_YarnSpinner_Finished`, if you want to compare.

1. A first line: Sign.yarn

Make a Yarn script called `Sign`:

```
title: Sign
---
Sign: Mossy Meadow Village. Population: 2 (and a lot of slimes).
===
```

- A Yarn script is made of **nodes**. Each one has a **`title:`**, then runs from **`---`** to **`===`**. The title is how the game finds it.
- A line is **`Name: text`**. The name before the colon shows in the box's name field.

Save it and select **Village** in the Project window: the Inspector lists the nodes it found, and shows any mistakes in red.

2. Starting a conversation: YarnTalkable

Make a folder `Assets/_game/Scripts/T11b_YarnSpinner` and create `YarnTalkable` in it:

```
// T11's Talkable, but the lines live in a Yarn script. Walk up, press Interact, and the
// Dialogue Runner plays the node with this object's name: lines, choices, variables and
// commands all come from the .yarn file. The Dialogue System prefab's Line Advancer moves
// through the lines.
using UnityEngine;
using UnityEngine.InputSystem;
using Yarn.Unity;

public class YarnTalkable : MonoBehaviour
{
    [SerializeField] private InputActionReference interactAction;
    [SerializeField] private DialogueRunner dialogueRunner;
    // The title of the node to start, from a .yarn file ("title: Blacksmith").
```

```
    [SerializeField] private string startNode = "Blacksmith";
    [SerializeField] private GameObject prompt;
    // Switched off while talking, so the player stands still (TopDownMovement and
    // MeleeAttack).
    [SerializeField] private MonoBehaviour[] disableWhileTalking;

    private bool playerInReach;
    private bool isTalking;
    // The press that finishes the last line mustn't start the conversation again.
    private int endedOnFrame = -1;

    private void Awake()
    {
        prompt.SetActive(false);
    }

    private void OnEnable()
    {
        interactAction.action.Enable();
        dialogueRunner.onDialogueComplete.AddListener(OnDialogueComplete);
    }

    private void OnDisable()
    {
        interactAction.action.Disable();
        dialogueRunner.onDialogueComplete.RemoveListener(OnDialogueComplete);
    }

    private void Update()
    {
        bool canTalk = playerInReach && !dialogueRunner.IsDialogueRunning &&
            Time.frameCount != endedOnFrame;
        prompt.SetActive(canTalk);

        if (canTalk && interactAction.action.WasPressedThisFrame())
        {
            isTalking = true;
            foreach (MonoBehaviour script in disableWhileTalking)
            {
                script.enabled = false;
            }
            // StartDialogue runs in the background; we don't need to wait for it here.
            _ = dialogueRunner.StartDialogue(startNode);
        }
    }

    // Called by the Dialogue Runner's On Dialogue Complete event when any conversation
    // ends.
    private void OnDialogueComplete()
    {
        endedOnFrame = Time.frameCount;
        if (!isTalking)
        {
            return;
        }

        isTalking = false;
        foreach (MonoBehaviour script in disableWhileTalking)
        {
```

```
            script.enabled = true;
        }
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            playerInReach = true;
        }
    }

    private void OnTriggerExit2D(Collider2D other)
    {
        if (other.CompareTag("Player"))
        {
            playerInReach = false;
        }
    }
}
```

- **Compare it with T11's Talkable.** The lines have gone; instead there's **`startNode`**, the title of a node. `dialogueRunner.StartDialogue(startNode)` plays it, and Yarn's prefab does all the showing.
- **`onDialogueComplete`** is an event on the Dialogue Runner. `AddListener` in `OnEnable` means `OnDialogueComplete` runs whenever any conversation ends, which gives the player back control.
- **`isTalking`** remembers whether *this* person started the conversation. Every YarnTalkable hears the event, but only the one that froze the player unfreezes it.
- **`endedOnFrame`** does the same job as T11's `IsBusy`: the E press that finishes the last line mustn't start the conversation again.
- `_ =` throws away what `StartDialogue` returns (a task you could wait for). We don't need to wait for it.

Add it to **People > Sign**. Set **Interact Action** to **Player/Interact**, drag **Dialogue System** into **Dialogue Runner**, type `Sign` in **Start Node**, and drag the Sign's own child **Prompt** into **Prompt**. For **Disable While Talking**, click **+** twice. These slots want a *component*, so right-click the **Player** in the Hierarchy and choose **Properties...**: a small Inspector window opens for it. Drag its **Top Down Movement** component (by its title) into **Element 0**. Do the same with **Player > Sprite** and drag its **Melee Attack** into **Element 1**.

Test it: press Play, walk up to the sign and press E. The sign's line types out in Yarn's box; press E to finish it and the box closes.

3. Yarn talks to C#: YarnInventoryCommands

The blacksmith needs to count your coins, take them and give you a key. Yarn can call C# for that. Create `YarnInventoryCommands` in the `T11b_YarnSpinner` folder:

```
// Lets Yarn scripts use the player's inventory (T09). A Yarn script can call:
//   coins()          a function that returns how many coins the player has
//   <<take_coins 3>> a command that takes coins away
//   <<give_key>>     a command that gives the player a gold key
// The [YarnFunction] and [YarnCommand] attributes tell Yarn Spinner about these methods,
// so the .yarn files can use them. They're static (they belong to the class, not to one
// object), which is why the inventory and items are kept in static fields too.
using UnityEngine;
using Yarn.Unity;
```

```
public class YarnInventoryCommands : MonoBehaviour
{
    [SerializeField] private Inventory playerInventory;
    [SerializeField] private ItemData coin;
    [SerializeField] private ItemData goldKey;

    private static Inventory inventory;
    private static ItemData coinItem;
    private static ItemData keyItem;

    private void Awake()
    {
        inventory = playerInventory;
        coinItem = coin;
        keyItem = goldKey;
    }

    [YarnFunction("coins")]
    public static int CountCoins()
    {
        return inventory.CountOf(coinItem);
    }

    [YarnCommand("take_coins")]
    public static void TakeCoins(int amount)
    {
        for (int i = 0; i < amount; i++)
        {
            inventory.Remove(coinItem);
        }
    }

    [YarnCommand("give_key")]
    public static void GiveKey()
    {
        inventory.Add(keyItem);
    }
}
```

- **A function gives Yarn a value; a command does something.** `[YarnFunction("coins")]` lets Yarn write `coins()` to ask how many coins you have. `[YarnCommand("take_coins")]` lets it write `<<take_coins 3>>`; the `3` becomes `amount`.
- **`static`** means the method belongs to the class, not to one object, so Yarn can call it without finding the object first. A static method can only use static fields, so `Awake` copies the Inspector slots into the static ones.
- **The C# uses T09's Inventory** (`CountOf`, `Remove`, `Add`). Yarn doesn't need to know how the inventory works.

Add it to **Dialogue System**. Drag the **Player** into **Player Inventory** (it has only one Inventory, so dragging the object is fine), and the **Coin** and **Gold Key** assets from `Assets/_game/ScriptableObjects/Items/` into **Coin** and **Gold Key**.

4. Choices and memory: Blacksmith.yarn

Make a Yarn script called `Blacksmith`:

```
title: Blacksmith
---
// Variables start with $. Declaring one gives it a type and a starting value.
// The Dialogue Runner remembers them for the whole game, so other nodes can read
// them too.
<<declare $metBlacksmith = false>>
<<declare $boughtKey = false>>

<<if $metBlacksmith == false>>
    Blacksmith: Ah, a visitor! I'm Brann. Mind the anvil, it bites.
    <<set $metBlacksmith to true>>
<<else>>
    Blacksmith: Back again?
<<endif>>

<<if $boughtKey>>
    Blacksmith: That key opens the gate to the pen. Off you go.
<<else>>
    Blacksmith: I've a spare gold key. Yours for 3 coins.
    // Each -> line is a choice for the player. The indented lines under it run if
    // it's picked.
    -> I'll buy it.
        // coins() is a function written in C# (YarnInventoryCommands): it counts the
        // coins you carry.
        <<if coins() >= 3>>
            // take_coins and give_key are commands, also written in C#.
            <<take_coins 3>>
            <<give_key>>
            <<set $boughtKey to true>>
            Blacksmith: Pleasure doing business. That key opens the gate to the pen.
        <<else>>
            Blacksmith: You've only got {coins()}. Come back with 3.
        <<endif>>
    -> Where do I find coins?
        Blacksmith: Lying about the square, mostly. Walk over them to pick them up.
    -> Bye.
        Blacksmith: Come back when you're rich!
<<endif>>
===
```

- **Variables** start with `$`. **`<<declare>>`** creates one with a starting value, **`<<set ... to ...>>`** changes it, and the Dialogue Runner remembers it for the rest of the game. That's how he knows he's met you.
- **`<<if>>`, `<<else>>`, `<<endif>>`** work like C#'s `if`: only one branch runs. The lines inside are indented so you can see which branch they belong to.
- **`->` lines are choices.** The game shows them as buttons, and the indented lines under the one you pick run.
- **`coins()`** calls your C# function. In a line, **`{coins()}`** puts its value into the text. **`<<take_coins 3>>`** and **`<<give_key>>`** run your C# commands.
- `//` starts a comment, as in C#: the game ignores it.

Add it: add **YarnTalkable** to **People > Blacksmith**, with the same slots as the Sign but **Start Node** `Blacksmith` and the Blacksmith's own **Prompt**.

Test it: press Play and talk to the blacksmith. Pick a choice with the **up and down arrows** (or the mouse, or the gamepad) and confirm with **Enter** (or the gamepad's South button). Try to buy the key with no coins, then collect 3 and buy it: 3 coins leave the inventory row and a key arrives. Talk to him again: "Back again?". Walk into the **Locked Door**: it opens.

5. Shared memory: Villager.yarn

Make a Yarn script called `Villager`:

```
title: Villager
---
// $boughtKey is declared in Blacksmith.yarn. Variables are shared between all the nodes.
<<if $boughtKey>>
    Villager: You bought Brann's key? The gate to the pen is just over there.
<<else>>
    Villager: Brann the blacksmith sells keys. He's very fond of coins.
    Villager: Want a tip?
    -> Yes please.
        Villager: Coins lie around the square. Walk over them to pick them up.
    -> No thanks.
        Villager: Suit yourself!
<<endif>>
===
```

The villager reads **`$boughtKey`**, which the blacksmith's node sets. Variables are shared by every node in the Yarn Project, so the whole village can react to what you did.

Add it: add **YarnTalkable** to **People > Villager**, with **Start Node** `Villager` and the Villager's own **Prompt**.

Test it: talk to the villager before and after buying the key. The advice changes.

Check your work

- ☐ **Village** in the Project window lists the nodes **Sign**, **Blacksmith** and **Villager** with no errors.
- ☐ The **E** shows when you're close to someone, and hides while you're talking.
- ☐ The blacksmith introduces himself the first time and says "Back again?" after that.
- ☐ With fewer than 3 coins he tells you how many you have. With 3, the coins go and the key arrives.
- ☐ The villager's advice changes after you buy the key, and the key opens the pen.
- ☐ You can't walk or swing the sword while talking.
- ☐ Compare with `CRE132_T11b_YarnSpinner_Finished` if something behaves differently. Its `.yarn` files are in `Assets/_game/Dialogue/T11b/`.

Key points

- **Dialogue is data, written in a script file, not in C#.** A few functions and commands connect it to the game.
- **Variables let the world remember** what the player did, and every node can read them.
- **One Dialogue Runner and one set of UI serve every character.** Each character only says which node to start.

If something goes wrong

- **Lots of red errors about `Yarn.Unity` when the project opens:** the package didn't download. Check you're online and that Git is installed, then reopen the project.
- **"No node named Blacksmith" (or similar) in the Console:** the **Start Node** doesn't match the node's `title:` exactly, or the `.yarn` file isn't in `Assets/_game/Dialogue/T11b/`.
- **Village shows errors in the Inspector:** a typing mistake in a `.yarn` file. Check every `<<if>>` has an `<<endif>>`, every node ends with `===`, and the choice lines are indented.
- **"coins" or "take_coins" is unknown:** YarnInventoryCommands has an error, or the name in `[YarnFunction("coins")]` doesn't match the Yarn script.

- **NullReferenceException in YarnInventoryCommands:** its **Player Inventory**, **Coin** or **Gold Key** slot is empty.
- **The player can still walk while talking: Disable While Talking** has the wrong components in it. Use **Properties…** and drag the components themselves.
- **The choices never appear:** select **Dialogue System**, find its **Options Presenter**, and check **Option View Prefab** isn't empty (it should be the package's **Option Item**).

Try it

1. Add a third choice that only appears once you've met the blacksmith: `-> Tell me about the slimes. <<if $metBlacksmith>>`, with an answer indented under it.
2. Write a `<<heal 2>>` command that calls the player's `Health.Heal`, so the blacksmith patches you up for a coin.
3. Make the sign say something different once the pen is open (a `$penOpen` variable, set by a command that `LockedDoor` uses).
4. Give the blacksmith a random greeting with line groups (`=>`) or `<<once>>`.